

openGauss 数据库查询功能探索

一、探索内容

本实验通过阅读和调试 openGauss 的源码，深入理解 SQL 语句从文本到执行结果的完整处理流程。重点选取 SELECT、UPDATE 和 JOIN 三种典型操作，分为以下三个阶段：

语法解析阶段：通过分析语法定义和生成的解析器，理解词法分析、语法分析如何将 SQL 文本转换为抽象语法树（AST）；

查询规划阶段：跟踪优化器入口，观察如何在逻辑计划和物理计划之间进行转换，如何生成可行执行路径并进行成本估算；

执行阶段：从执行器主入口开始，逐步执行各类节点（如顺序扫描、哈希连接等），并在关键节点处记录数据在内存中如何流转，以及如何产生最终结果。

二、探索目标

掌握 openGauss 在语法解析、查询优化和执行三个核心模块中的关键代码位置和调用流程；

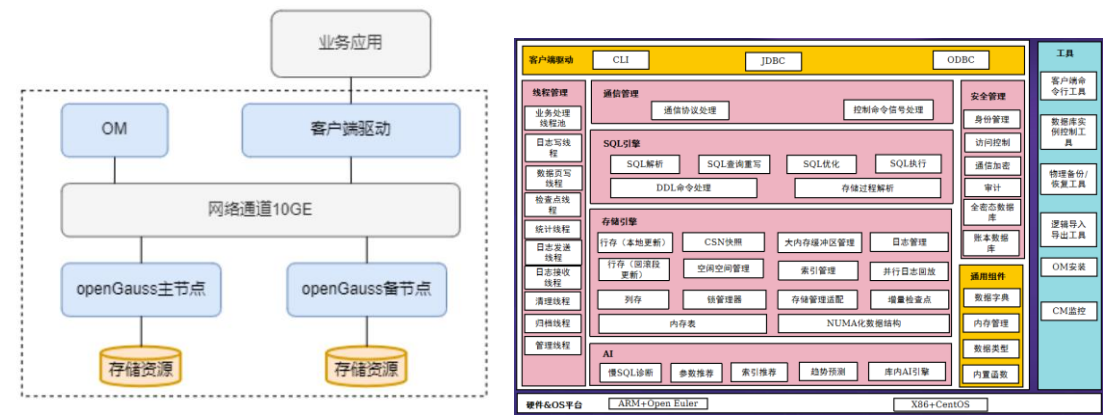
能够截取并注释 SELECT、UPDATE、JOIN 三种操作的关键函数调用链，清晰展现各阶段的输入输出和数据结构演变；

通过对比 openGauss 与 MySQL、PostgreSQL、SQLite 在进程模型、存储架构、扩展性与适用场景等方面的设计差异，加深对主流数据库系统架构的理解，为后续性能调优和二次开发奠定基础。

三、openGauss 系统架构总览

1. 逻辑架构

openGauss 的逻辑架构中主要包含了 OM（Operation Manager）、客户端驱动、openGauss 服务器、存储器等模块。

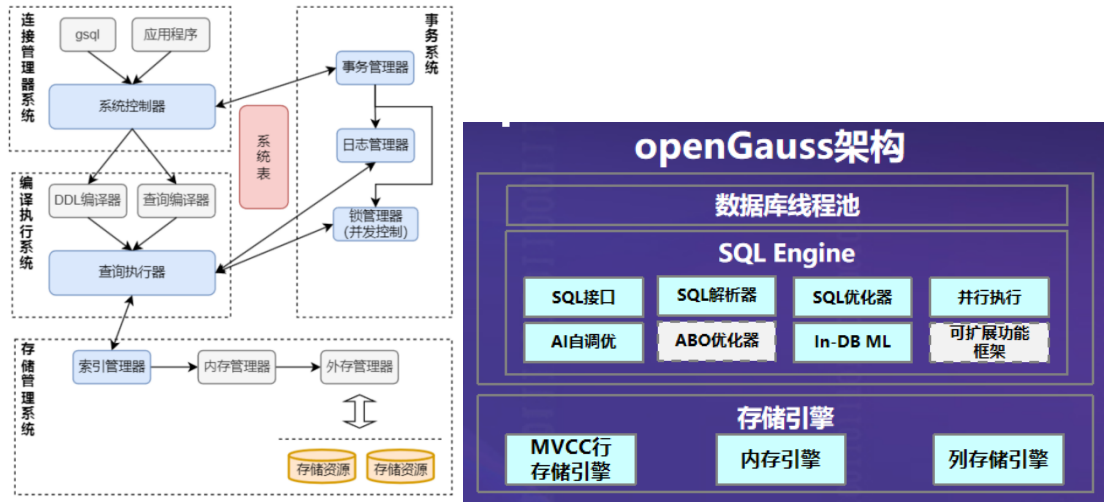


OM 在 openGauss 数据库系统中，指代运维管理模块，这是一个关键的组件，负责提供与 openGauss 数据库相关的日常运维操作和配置管理功能。客户端驱动则负责接收来自业务应用的查询请求，然后与 openGauss 实例进行通信，将查询下发至 openGauss 实例中进行具体的执行，而后将收到的查询执行结果汇报给业务应用。10GE (Gigabit Ethernet) 即 10 吉比特以太网，其规范了以 10Gbit/s 的速率传输数据的以太网，极大地增强了数据传输速度和网络性能。这一突破性技术不仅提升了局域网内部的数据传输效率，还因其高带宽特性得以在城域网乃至广域网中得到广泛应用，实现了更大范围内的端到端以太网连接和数据传输。

openGauss 节点负责处理并返回用户提交的数据查询任务，管理包括行存储引擎、列存储引擎以及内存表在内的多种类型业务数据的存储与访问。openGauss 支持灵活的部署模式，包括单机部署以及一主多备的集群部署架构，其中主节点负责处理主要事务及查询请求，备节点则可以进行实时同步，并在主节点故障时快速切换接管服务，保证系统的高可用性和稳定性。同时，openGauss 还支持备机可读功能，进一步提高资源利用率和系统扩展能力。最后，提到的存储资源是指服务器内部或外部用于持久化存储数据库数据的各种物理或虚拟存储设备，如硬盘、固态硬盘 (SSD)、网络附加存储 (NAS) 或存储区域网络 (SAN)，这些存储资源为 openGauss 提供了可靠的数据存储基础，确保了数据库信息的安全保存和高效访问。

2. 体系架构

openGauss 的体系结构由连接管理、编译执行、事务系统和存储管理四个系统和系统表组成。



系统表在 openGauss 数据库中扮演着至关重要的角色，它是存储和管理所有与数据库结构元数据相关的核心区域。具体来说，系统表包含了对象及其属性的描述信息和自然语言含义、对象之间关系的描述信息、数据库的状态信息等关键信息。系统表作为 openGauss 数据库的心脏地带，为数据库管理系统提供了全面而详尽的内部视图，是数据库进行自我管理和优化的基础。系统表一般由系统统一维护，从而维护系统表信息的一致性，防止用户手动修改导致系统控制紊乱。用户可以通过系统视图查询系统表和访问数据库内部状态，系统视图在初始化数据库集群时通过读取脚本完成创建。

在连接管理系统中，openGauss 和其他数据库系统一样，支持用户通过客户端工具或应用程序使用特定协议（如 TCP/IP）建立到数据库服务器的连接。连接过程中会涉及用户名、密码的验证，以及 SSL/TLS 加密连接的支持，确保数据传输 13 电子科技大学硕士学位论文的安全性。用户可以通过 gsql、应用程序等客户端工具连接 openGauss。openGauss 拥有一个叫做 gsql 的自带客户端工具，其能够通过 SQL 命令语句与 openGauss 完成交互。其中应用程序支持 JDBC（Java Database Connectivity）、ODBC（OpenDatabase Connectivity）、Libpq 和 Psycopg 等驱动程序，这些驱动程序都是用于执行 SQL 语句的应用程序接口。另外，openGauss 提供连接池功能和并发控制，监控并管理连接的数量和回收策略，从而减少频繁创建和销毁连接、防止连接资源耗尽。

编译执行系统包括 DDL 编译器（Data Definition Language）、查询编译器、查询执行器。DDL 编译器主要负责解析、验证并转换创建、修改和删除数据库结构的简单 SQL 语句。而关于增删改查等复杂查询命令，在查询编译器中，openGauss 将对客户端传来的 SQL 语句依次进行词法分析、语法分析、语义分析、查询重写、查询优化，最终生成优化后的执行计划。在查询优化的过程中涉及到成本估算、索引选择、连接策略的选择以及并行执行策略的设计，从而减少文件 I/O 的开销和数据计算代价。最后，查询执行器将按照查询优化创建的执行计划，进行读取数据、筛选和运算、合并结果集等具体查询操作。

事务系统包括事务管理器、日志管理器和锁管理器。其中，事务管理器是事务系统的中枢，通过实现一个有限状态自动机接收上层模块的命令信号，然后根据当前事务所处的状态，决定事务下一步的具体执行过程。日志管理器通过事务提交日志和事务日志，分别记录了事务执行的状态和数据的变动过程，从而保证系统故障后能够通过这些日志恢复数据库的一致性和完整性。锁管理器实现了系统并发控制所需要的各种锁，从而实现了事务读阶段的多版本并发控制和事务写阶段的隔离级别。

存储管理系统包括索引、内存、外存三种管理器和存储资源。其中索引管理器主要负责索引的创建、修改、删除和使用。openGauss 的索引管理能够监控索引的使用效率，通过分析相关信息从而判断是否充分利用索引并优化索引结构。内存管理器负责管理和优化 openGauss 内部以及与操作系统之间的内存资源分配，包括共享内存管理、缓冲池管理、jemalloc 内存分配、内存统计与监控等。外存管理器负责对磁盘存储空间的管理，确保数据在磁盘上的高效组织和持久化存储，包括存储空间的分配与回收、缓存管理协作、故障恢复与备份等。

3. 架构设计特点与优势

从源码结构和模块划分来看，openGauss 的架构设计体现了以下特点：

高度模块化（Modularity）：系统被划分为多个相对独立的模块（如解析、优化、执行、存储、事务等），每个模块有清晰定义的职责和接口。这种设计有利于并行开发、单独测试、维护和未来的功能扩展。CMake 构建系统也反映了这种模块化的组织方式。

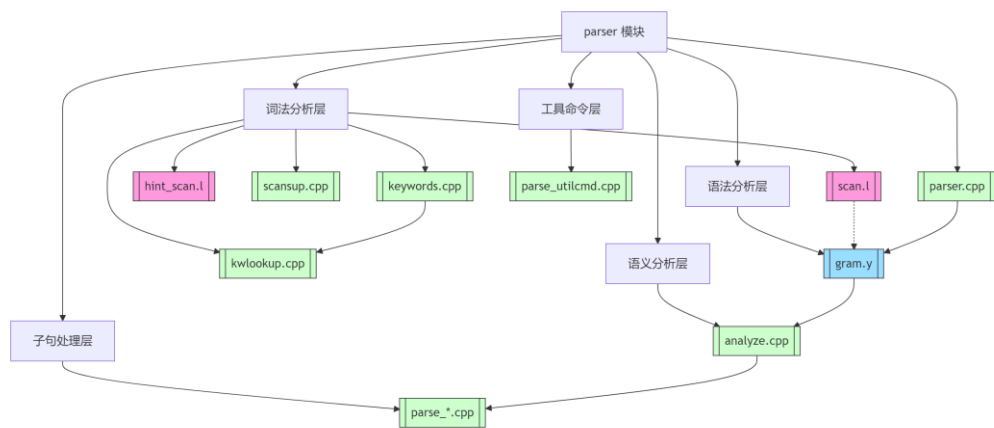
清晰的分层结构（Layered Structure）：架构层次分明，从客户端接口、连接管理、SQL 处理（编译与执行）、事务管理到最底层的存储管理，各层之间通过既定接口交互。

可移植性 (Portability): 系统在设计上考虑了跨不同操作系统的可移植性, 将平台相关的代码进行了抽象和封装。

功能完备性 (Comprehensive Features): 覆盖了现代关系型数据库所需的绝大部分核心功能, 包括复杂的 SQL 解析与优化、ACID 事务保证、MVCC (通常在 transam 和 access 中实现)、WAL、多种索引支持、备份恢复工具等。

四、openGauss 查询语句执行过程拆解

1. 输入语句解析



1.1 词法解析

语句解析要先进行词法解析和语法解析。

对于 openGauss, SQL 文本的正则规则定义在 scan.l / hint_scan.l 文件中, 编译时再由 flex 生成词法分析文件, 除此之外, scansup.cpp 用于处理输入串中的 escape、unicode 等预处理。关键字识别上, 由 keywords.cpp 识别 SQL 保留字表, kwlookup.cpp 将扫描到的标识符映射到关键字 Token。

总的词法分析流程: raw_parser → core_yylex_init → scanner_init (scan.l) → 字符串 / 注释处理 (scansup) → 识别标识符 / 关键字 (kwlookup)。

1.2 语法解析

接下来是语法解析。gram.y 定义了 SQL 语法规则, 并由 bison 生成 gram.cpp。Parser.cpp 中, raw_parser 调用 Bison 生成的 raw_parse, 输出一系列 RawStmt (“原始”解析树)。

1.3 原始解析树节点构建

parse_node.cpp 提供 make*Node、copyObject 等通用节点构造/拷贝函数。

parsenodes_common.h / parsenodes.h (头文件) 定义 RawStmt、A_Expr、SelectStmt 等 AST 数据结构。

1.4 语义分析&解析树转换

文件: analyze.cpp

```
// parse_analyze 将 RawStmt → Query，并检查权限/类型等
Query* parse_analyze(Node *parseTree, const char *sourceText, ...);
```

内部会调用各模块：

```
parse_relation.cpp: FROM、表/别名
parse_target.cpp: SELECT 列表
parse_clause.cpp: WHERE/GROUP BY/HAVING/ORDER BY
parse_expr.cpp: 算术/逻辑表达式、子查询
parse_oper.cpp: 操作符解析、重载查找
parse_func.cpp: 函数调用、运算符变换
parse_agg.cpp: 聚合函数
parse_type.cpp / parse_coerce.cpp: 类型字面值 & 隐式强转
parse_collate.cpp: 排序规则
parse_param.cpp: 参数引用（预处理语句）
parse_cte.cpp: WITH 子句
parse_merge.cpp: MERGE 命令
parse_hint.cpp: 优化器 Hint 处理
parse_compatibility.cpp: 兼容性语法（Oracle/昇腾）
parse_utilcmd.cpp: 非优化命令（CREATE/ALTER/DROP/EXPLAIN...）
```

每个 `parse_*.cpp` 中，都是针对特定 SQL 结构做 `transform*()`，最终构造一个完整的 Query 对象。

2. 查询语句优化

优化文件大纲：

```
src/gausskernel/optimizer
├─ CMakeLists.txt
├─ Makefile
├─ commands/           # 各种 DDL/DML 命令在优化阶段的支持
├─ geqo/               # GEQO（遗传算法）查询优化
├─ path/               # 路径枚举与代价估算
├─ plan/               # 将 Path 转换成 Plan 树
├─ prep/               # 预处理（prepared stmt、Bind/Execute 支持）
├─ rewrite/            # 规则/视图重写
├─ util/               # 公共辅助函数（子句处理、RelOptInfo 构造等）
├─ aioptimizer/        # AI 驱动统计与选择性估算
├─ gplanmgr/           # 执行计划管理与缓存
├─ sqladvisor/         # SQL 调优建议系统
├─ sqlpatch/           # SQL 修补（patch）支持
├─ statistics/         # 单节点下的统计信息扩展
```

2.1 重写阶段

解析得到的 query 对象进行视图展开 & 规则重写。`rewriteHandler.cpp` 负责规则合法性检查（`check_rules`）和调用核心重写；`rewriteManip.cpp` 进行具体的视图展开、规则合并逻辑。

`rewriteRemove.cpp` 删除规则并更新。

2.2 规划阶段

对每条重写后的 Query 调用 planner，构建 PlannerInfo，调用 query_planner 枚举所有可行 Path

2.3 路径枚举并选择最优 path

query_planner 内部依次调用各接口，生成基表及连接 Path，各模块 (in dxpath.cpp、joinpath.cpp) 估算每种路径的代价

收集所有 Path 后，在 planner.cpp 中以成本最小策略选出

2.4 生成执行计划树

递归调用 create*_plan 构建 Plan 节点

```
/*  
 * create_plan_recurse  
 * Recursive guts of create_plan().  
 */  
static Plan* create_plan_recurse(PlannerInfo* root, Path* best_path)  
{
```

2.5 AI 辅助 & 计划管理

若启用 AI 模块，在 aiselectivity 阶段使用机器学习模型调整成本，gp lanmgr 对生成后的 PlannedStmt 做缓存

通过以上各模块分工和流程，openGauss 实现了对 Query 查询对象的优化。

3. 查询执行流程

关键文件夹及模块：

executor 目录 (src\gausskernel\runtime\executor)：

该目录中包含了绝大多数基于行式执行的计划节点实现，

比如：execProcnode.cpp：提供了 ExecInitNode、ExecProcNode、ExecEndNode 框架的分发入口。各种具体的节点实现文件（例如 nodeNestloop.cpp、nodeSeqscan.cpp、nodeIndexscan.cpp）。

lightProxy.cpp：用于轻量级查询转发，如对特定命令（例如 UPSERT）或路由查询的预检查、调度及特殊规则处理。

vecexecutor 目录 (src\gausskernel\runtime\vecexecutor)：

内含面向向量化执行的节点代码，例如向量窗口聚合 (vecwindowagg.cpp)、向量远程查询等。

codegen 目录 (src\gausskernel\runtime\codegen)：

包含针对 LLVM IR 编译的代码生成实现 (gscodegen.cpp)，用于生成机器码以进一步提升表达式或部分操作的执行性能。

3.1 初始化阶段

本阶段在 ExecutorStart() 中完成，该函数作为执行器入口，调用 ExecInitNode() 递归初始化整个计划树，各节点初始化函数（如 ExecInitNestLoop、ExecInitSeqScan、ExecInitIndexScan 等）负责分配表达式环境 (ExprContext)、设置扫描句柄等。

3.2 执行阶段：

该阶段会对每个计划节点进行处理，实际调用各具体节点的执行函数，如：

ExecNestLoop() (nodeNestloop.cpp) 处理嵌套循环连接。

ExecIndexScan()、IndexRecheck() (nodeIndexscan.cpp) 处理基于索引的扫描和资格检查。

ExecBitmapHeapScan() 与 BitmapHeapRecheck() (nodeBitmapHeapscan.cpp) 用于位图扫描场景。

在向量化执行路径中，ExecVecWindowAgg()、ExecInitVecRemoteQuery() 等在 vecexecutor 下相应文件中实现，用于批量数据处理。

3.3 结束阶段：

该阶段在 ExecutorEnd() 与 ExecutorFinish() 中实现。分别调用 ExecEndNode() 系列函数（如 ExecEndNestLoop、ExecEndSeqScan、ExecEndBitmapHeapScan、ExecEndWindowAgg 等）释放各节点占用的内存、关闭扫描关系、清空表达式上下文等。

3.4 其他模块：

openGauss 执行器中还有一些其他的模块：

代码生成模块：在 gscodegen.cpp 中，函数如 CodeGenPassThreshold() 用于判断是否满足编译机器码的阈值，CodeGenThreadReset() 和 CodeGenThreadTearDown() 用于管理生成的代码和线程状态。

轻量代理 (lightProxy)：函数如 lightProxy::checkRouterQuery() 和 lightProxy::checkLightQuery() (见 lightProxy.cpp) 对部分特殊查询进行预处理和路由判断，支持一些特定场景下的快速调度。

通过各模块之间的分层协作，openGauss 能够高效地执行复杂查询，同时利用缓存、向量化、代码生成等技术进一步提升整体性能。

4. 查询后存储流程

关键目录及模块：

storage/buffer：管理缓冲区、维护 BufferTable 及执行脏页标记、刷写操作。

bufmgr.cpp：包含如 ReadBuffer()、ReleaseBuffer()、MarkBufferDirty() 等接口。buf_table.cpp：负责 BufferTag 到缓存索引的映射和查找。

storage/access/transam: 负责管理 WAL 日志，保证数据持久性与 crash recovery。

xlog.cpp: 实现了 WAL 日志写入、读取、校验等操作，函数如 WriteXlogToShareStorage、FindLastRecordCheckInfoOnShareStorage。

storage/xlog_share_storage: 提供对共享存储 WAL 的管理和数据同步，将本地 WAL 日志复制到共享存储以便灾备、同步。

xlog_share_storage.cpp: 包含 DoXLogCopyFromLocal、XLogOverwriteFromLocal、LockNasWriteFile、UnlockNasWriteFile 等函数，管理 WAL 的复制、控制信息的校验与更新。

storage/cstore: 针对列存储表的写入、扫描及数据维护，支持批量操作和内存分配。

cstore_rewrite.cpp: 中的 HeapBulkInsert::HeapBulkInsert 函数用于初始化批量插入。**cstore_am.cpp:** 提供 CStoreBeginScan、CStoreGetNextBatch 及 CStoreEndScan 等接口，实现对列存储数据的扫描与批处理操作。

storage/access: 包括行存储和索引更新等。

NBTree/UBTree: 实现 B+Tree 索引的插入、调整操作，相关函数如 _bt_newroot（在 nbtinsert.cpp）和 UBTreeInsertOnPage（在 ubtinsert.cpp）。其他存取子模块（heap、gin、gist 等）：分别实现对应数据结构和索引的数据存储与回放逻辑。

4.1 缓存更新与脏页标记

数据修改首先在共享内存中的 Buffer（缓存）更新，缓冲区由 storage/buffer 模块管理。相关函数在 bufmgr.cpp 中实现。

4.2 WAL 日志记录（预写式日志）

为保证数据一致性，所有修改操作都会记录到 WAL 日志中。这部分代码主要位于 storage/access/transam/xlog.cpp 及其相关模块中。写入 WAL 的工作进一步会调用写入共享存储的接口，例如：WriteXlogToShareStorage 和在 xlog_share_storage/xlog_share_storage.cpp 中的操作。

4.3 磁盘刷写与控制信息更新

缓存中的脏页根据需要由后台线程通过 Buffer Manager 调度写入磁盘，同时 WAL 日志会定期 flush 到磁盘。对共享存储（如 standby 情况）来说，会更新控制信息（CTL）。

4.4 索引更新及列存储操作

对于表数据和索引来说，更新操作还涉及基于行存储（如 Heap、NBTree、UBTree）和列存储（CStore）的相应写入过程。

行存储相关函数见 access/nbtree/nbtinsert.cpp 和 access/ubtree/ubtinsert.cpp

列存储（CStore）方面，执行数据写入、批量插入、扫描和维护由 storag

e/cstore 下多个文件实现。例如：HeapBulkInsert 构造函数在 cstore_rewrite.cpp 中实现；扫描操作由 cstore_am.cpp 中的 CStoreBeginScan、CStoreGetNextBatch 及 CStoreEndScan 完成。

五、openGauss 与其他数据库架构对比

	openGauss	PostgreSQL	MySQL
进程模型	多进程 + MPP 分布式	多进程 MVCC	多线程/多进程（随存储引擎而异）
存储架构	行存+列存混合	行存	插件化 Storage Engine (InnoDB 等)
扩展性	高可用集群管理、 分布式事务、列存插件	扩展模块丰富	插件化引擎、主从复制、Group Replication
适用场景	OLTP/OLAP 混合、 企业级分布式	OLTP 单机、 大数据分析	Web 应用、高并发读写

六、查询流程优势与局限性

在深入分析 openGauss 源码后，结合关键代码片段，我们对其查询流程的优势与局限性作如下详尽探讨。

1. 优势分析

1.1 解析器与执行器解耦，便于扩展新语法

原理：

Bison/Yacc 负责将 SQL 文本转换成抽象语法树（AST），执行器只关注 AST 节点的执行，二者通过中间节点 Query 进行交互。解析结束后，raw_parser.c 调用 parse_analyze() 生成 Query 结构；执行阶段的入 exec_simple_query() 则遍历该结构，无需修改执行器代码即可支持新语法。

优势：

- 增函数、语法或关键字时，只需修改.y 文件，减少跨模块改动；
- 免语法变更导致执行逻辑错乱，提高维护性。

1.2 基于代价的优化器动态选择执行路径

原理：

针对同一逻辑查询，优化器生成多种路径（SeqScan、IndexScan、HashJoin、NestLoop 等），通过成本模型比较并选出最优。同时，EXPLAIN (ANALYZE, BUFFERS) 与 optimizer_trace 可输出每条路径的成本计算过程，便

于调优。

优势：

- 能自动选择最优执行方案，应对多样化数据分布；
- 插件式成本模型支持自定义算子成本计算，灵活扩展。

1.3 可插拔的存储与执行节点设计

原理：

openGauss 将存储层与执行层解耦，执行节点（`Plan`/`PlanState`）通过统一的接口注册到 Executor。

优势：

- 支持在不改动核心执行框架的前提下，注册新算子或存储引擎；
- 清晰的模块边界提升了并行开发效率。

2. 局限性发现

2.1 线程查询处理易产生锁竞争

在复杂查询或高并发 DDL 场景下，Session 线程会在轻量级锁或互斥量上排队，导致吞吐下降。

影响：大表 JOIN 或大规模 UPDATE 时，多线程对共享 Catalog、缓冲区管理器的争抢明显，整体查询延迟增大。

2.2 内存分配策略简单，频繁分配开销高

在顺序扫描大表或构建大型 HashJoin 时，MemoryContextAlloc 被频繁调用，分配与回收成本显著。AllocSetContextAlloc 在默认 AllocSet 中按块分配，缺乏针对大对象的专用缓存，导致碎片化和锁竞争。

影响：长事务或大数据量扫描时，GC 清理与分配压力大，抑制并发查询效率。

2.3 执行模型为逐行（tuple-at-a-time），缺乏向量化支持

Executor 针对每行数据调用一次 ExecProcNode，函数调用开销无法通过批量处理弥补。

影响：大规模数据聚合、扫描场景下，CPU 利用率低于向量化数据库，吞吐率受限。

以上内容结合了源码定位与执行流程，从模块解耦、优化能力、扩展性等方面肯定了 openGauss 的优势，同时也指出了锁竞争、内存管理与执行模型三方面的瓶颈，为后续性能优化和功能增强提供了明确方向。

七、总结

openGauss 的分层架构将 SQL 处理流程明确划分为解析、优化和执行三个阶段，各阶段通过中间结构体（如 Query、PlannedStmt）衔接，保证了功能模块的独立性与可替换性。开发者如果需要新增语法，只需在解析层的 Bison 规则文件中定义新规则，不会影响执行层代码，大大降低了二次开发和维护成本。

在优化阶段，基于代价模型的优化器会生成多种执行路径，并动态选择最优方案；可插拔的算子接口意味着引入新存储引擎或自定义算子时，仅需注册相应

附录

参考文献:

- [1]杨瑞鑫. 基于 openGauss 的 Parquet 查询引擎优化研究与实现[D]. 电子科技大学, 2024. DOI:10.27005/d.cnki.gdzku.2024.003072.
- [2]广州海量数据库技术有限公司. OpenGauss 中实现 percentile_cont 分析函数的方法:CN202111551141.9[P]. 2025-01-14.
- [3]李国良, 王江, 陈国. openGauss:An Enterprise-Grade Open-Source Database System[J]. 计算机科学技术学报(英文版), 2024, 39(5):1007-1028. DOI:10.1007/s11390-024-4302-2.

组员分工:

2351520 毛星博: openGauss 系统架构总览

2354366 徐华炫: openGauss 查询语句执行过程拆解

2352841 刘佳鑫: openGauss 查询语句执行过程拆解

2353599 冯灏然: openGauss 与其他数据库架构对比+查询流程优势与局限性分析